Day 5 Task Workflow — Saturday, Mar 1, 2026

> **Sprint:** 1 (Foundation) | **Phase:** Docker Stack Verification & Backend Integration
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Docker stack operational, database connected, Redis verified, backend API accessible, design system components built

---

Day 5 Objective

Complete the deferred infrastructure verification and backend integration tasks so that by end of day:

- Docker Compose stack running (PostgreSQL + Redis)
- Database migrations applied successfully
- Redis pub/sub verified with live connection
- Backend API accessible at `/docs` with all endpoints
- Frontend connecting to backend API
- Design system components: GlassPanel, StatusBadge, MetricCard, DataTable
- Health check endpoint returning full system status

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope.

Requirement	Source Document	Section
Docker Compose stack	MASTER_DOC_PART2	§3.2 (Service Definitions)
PostgreSQL 16	MASTER_DOC_PART2	§8.1 (Technology Stack)
Redis 7	MASTER_DOC_PART2	§8.1
Alembic migrations	MASTER_DOC_PART5	§6.1 (Dev Setup)
FastAPI auto-docs	MASTER_DOC_PART2	§5.1 (API Architecture)
Glassmorphism UI	MASTER_DOC_PART3	§1.4 (Component Patterns)
Design system CSS	MASTER_DOC_PART3	§1.2 (Color System)
Health check endpoint	MASTER_DOC_PART2	§5.1 (`/system/health`)

Task Breakdown

TASK 1 — Docker Compose Stack Verification ⬤

Time Est: 20 min | **Priority:** ⬤ Critical

Verify Docker Desktop is running and start PostgreSQL + Redis containers.

1.1 Start Docker Stack

```
docker-compose -f docker-compose.dev.yml up -d postgres redis
```

1.2 Verify Containers Running

```
docker-compose ps
```

Expected Output:

```
NAME                    STATUS              PORTS
threatmatrix-postgres   Up                  0.0.0.0:5432->5432/tcp
threatmatrix-redis      Up                  0.0.0.0:6379->6379/tcp
```

1.3 Verify PostgreSQL Health

```
docker-compose exec postgres pg_isready -U threatmatrix
```

Expected Output:

```
localhost:5432 - accepting connections
```

1.4 Verify Redis Health

```
docker-compose exec redis redis-cli ping
```

Expected Output:

```
PONG
```

Verification:

Check	Command	Expected
Containers running	`docker-compose ps`	2 containers Up
PostgreSQL ready	`pg_isready`	accepting connections

Check	Command	Expected
Redis responding	`redis-cli ping`	PONG
Ports accessible	`netstat -an | grep 5432`	LISTENING
Ports accessible	`netstat -an | grep 6379`	LISTENING

TASK 2 — Database Connection Test 🟤

Time Est: 15 min | **Priority:** 🟤 Critical

Verify Python can connect to PostgreSQL and execute queries.

2.1 Test Database Connection

```
cd backend && python -c "
import asyncio
from app.database import engine
from sqlalchemy import text

async def test():
    async with engine.connect() as conn:
        result = await conn.execute(text('SELECT version()'))
        version = result.scalar()
        print('[OK] PostgreSQL version:', version)

        # Check if tables exist
        result = await conn.execute(text(\"\"\"
            SELECT table_name FROM information_schema.tables
            WHERE table_schema = 'public'
            ORDER BY table_name
        \"\"\"))
        tables = [row[0] for row in result]
        print('[OK] Tables found:', len(tables))
        for t in tables:
            print('  -', t)

asyncio.run(test())
"
```

Expected Output:

```
[OK] PostgreSQL version: PostgreSQL 16.x ...
[OK] Tables found: 10
  - alerts
  - audit_log
  - capture_sessions
  - llm_conversations
  - ml_models
```

```
    - network_flows
    - pcap_uploads
    - system_config
    - threat_intel_iocs
    - users
```

2.2 Run Alembic Migrations

```
cd backend && alembic upgrade head
```

Expected Output:

```
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade -> 20260226_000000,
initial_schema
```

2.3 Verify Migration Applied

```
cd backend && alembic current
```

Expected Output:

```
20260226_000000 (head)
```

Verification:

Check	Command	Expected
Connection works	Python test script	`[OK] PostgreSQL version:`
Tables exist	Python test script	`[OK] Tables found: 10`
Migration applied	`alembic current`	`20260226_000000 (head)`
Schema correct	`\dt` in psql	All 10 tables listed

TASK 3 — Redis Connection & Pub/Sub Test 🟤

Time Est: 15 min | **Priority:** 🟤 Critical

Verify Redis connection and pub/sub functionality.

3.1 Test Redis Connection

```
cd backend && python -c "
import asyncio
import redis.asyncio as redis

async def test():
    r = redis.from_url('redis://localhost:6379')

    # Test basic operations
    await r.set('test_key', 'hello')
    val = await r.get('test_key')
    print('[OK] Redis get/set:', val.decode())
    await r.delete('test_key')

    # Test ping
    pong = await r.ping()
    print('[OK] Redis ping:', pong)

    # Test pub/sub
    pubsub = r.pubsub()
    await pubsub.subscribe('test_channel')
    print('[OK] Redis pub/sub: subscribed to test_channel')
    await pubsub.unsubscribe('test_channel')

    await r.close()
    print('[OK] Redis connection closed')

asyncio.run(test())
"
```

Expected Output:

```
[OK] Redis get/set: hello
[OK] Redis ping: True
[OK] Redis pub/sub: subscribed to test_channel
[OK] Redis connection closed
```

3.2 Test RedisManager Class

```
cd backend && python -c "
import asyncio
from app.redis import RedisManager, CHANNEL_FLOWS_LIVE, CHANNEL_ALERTS_LIVE,
CHANNEL_SYSTEM_STATUS

async def test():
    manager = RedisManager('redis://localhost:6379')
```

```
    await manager.connect()

    # Health check
    health = await manager.health_check()
    print('[OK] Redis health:', health)

    # Cache operations
    await manager.set('cache_test', 'cached_value', ex=60)
    val = await manager.get('cache_test')
    print('[OK] Redis cache:', val)
    await manager.delete('cache_test')

    # JSON operations
    await manager.set_json('json_test', {'status': 'ok', 'count': 42})
    data = await manager.get_json('json_test')
    print('[OK] Redis JSON:', data)
    await manager.delete('json_test')

    await manager.disconnect()
    print('[OK] RedisManager test complete')

asyncio.run(test())
"
```

Expected Output:

```
[OK] Redis health: {'status': 'connected', 'latency_ms': 0.xxx}
[OK] Redis cache: cached_value
[OK] Redis JSON: {'status': 'ok', 'count': 42}
[OK] RedisManager test complete
```

Verification:

Check	Command	Expected
Redis accessible	Python test	`[OK] Redis get/set: hello`
Pub/Sub works	Python test	`[OK] Redis pub/sub: subscribed`
RedisManager works	Python test	`[OK] RedisManager test complete`
Health check	Python test	`{'status': 'connected'}`
Channels defined	Import check	`CHANNEL_FLOWS_LIVE`, `CHANNEL_ALERTS_LIVE`, `CHANNEL_SYSTEM_STATUS`

TASK 4 — Backend API Verification ⬛

Time Est: 20 min | **Priority:** ⬛ Critical

Start the backend server and verify all endpoints are accessible.

4.1 Start Backend Server

```
cd backend && uvicorn app.main:app --reload --port 8000
```

4.2 Verify Health Endpoint

```
curl http://localhost:8000/api/v1/system/health
```

Expected Output:

```
{
  "status": "healthy",
  "version": "0.1.0",
  "services": {
    "database": {"status": "connected"},
    "redis": {"status": "connected", "latency_ms": 0.xxx}
  }
}
```

4.3 Verify OpenAPI Docs

Open browser: http://localhost:8000/docs

Expected: Swagger UI showing all endpoints:

- `/api/v1/auth/*` (5 endpoints)
- `/api/v1/flows/*` (6 endpoints)
- `/api/v1/alerts/*` (5 endpoints)
- `/api/v1/system/*` (3 endpoints)

4.4 Verify API Routes

```
cd backend && python -c "
from app.main import app
routes = []
for route in app.routes:
    if hasattr(route, 'path'):
        routes.append(route.path)
routes.sort()
print('Total routes:', len(routes))
for r in routes:
```

```
    print(' ', r)
"
```

Expected Output:

```
Total routes: 19+
  /api/v1/alerts/
  /api/v1/alerts/{alert_id}
  /api/v1/alerts/{alert_id}/assign
  /api/v1/alerts/{alert_id}/status
  /api/v1/alerts/stats
  /api/v1/auth/login
  /api/v1/auth/logout
  /api/v1/auth/me
  /api/v1/auth/refresh
  /api/v1/auth/register
  /api/v1/flows/
  /api/v1/flows/protocols
  /api/v1/flows/search
  /api/v1/flows/stats
  /api/v1/flows/top-talkers
  /api/v1/flows/{flow_id}
  /api/v1/system/config
  /api/v1/system/health
  /api/v1/system/metrics
  /docs
  /openapi.json
  /redoc
```

Verification:

Check	Command	Expected
Server starts	`uvicorn`	No errors
Health endpoint	`curl /api/v1/system/health`	`{"status": "healthy"}`
Database connected	Health response	`"database": {"status": "connected"}`
Redis connected	Health response	`"redis": {"status": "connected"}`
OpenAPI docs	Browser `/docs`	Swagger UI loads
All routes mounted	Python check	19+ routes

TASK 5 — Frontend-Backend Integration Test ⚪

Time Est: 15 min | **Priority:** ⚪ Medium

Verify frontend can connect to backend API.

5.1 Start Frontend Dev Server

```
cd frontend && npm run dev
```

5.2 Verify API Client Configuration

Check `frontend/lib/api.ts`:

- Base URL: `http://localhost:8000`
- Fetch wrapper with auth headers
- Error handling

5.3 Test API Connection from Browser

Open browser console at http://localhost:3000 and run:

```
fetch('http://localhost:8000/api/v1/system/health')
  .then(r => r.json())
  .then(data => console.log('[OK] Backend connected:', data))
  .catch(err => console.error('[FAIL] Backend connection:', err))
```

Expected Output:

```
[OK] Backend connected: {status: "healthy", version: "0.1.0", ...}
```

5.4 Verify CORS Configuration

Check `backend/app/main.py` for CORS middleware:

- Allow origins: `http://localhost:3000`
- Allow methods: `GET, POST, PUT, PATCH, DELETE, OPTIONS`
- Allow headers: `*`

Verification:

Check	Command	Expected
Frontend starts	`npm run dev`	No errors
API client configured	Read `api.ts`	Base URL correct
CORS enabled	Backend check	Middleware present
Connection works	Browser console	`[OK] Backend connected`

TASK 6 — Design System Components 🟡

Time Est: 60 min | **Priority:** 🟡 Medium

Create reusable design system components per MASTER_DOC_PART3 §1.4.

6.1 GlassPanel Component Enhancement

Verify `frontend/components/shared/GlassPanel.tsx` has:

- `backdrop-filter: blur(12px)`
- `background: rgba(255, 255, 255, 0.03)`
- `border: 1px solid rgba(255, 255, 255, 0.06)`
- `border-radius: 12px`
- Optional `title` prop for panel header

6.2 StatusBadge Component

Verify `frontend/components/shared/StatusBadge.tsx` has:

- Severity levels: critical, high, medium, low, info
- Color mapping per design system
- Pulse animation for critical status

6.3 MetricCard Component

Verify `frontend/components/war-room/MetricCard.tsx` has:

- Animated counter (number increment animation)
- Sparkline chart integration
- Label and value display
- Cyan accent color

6.4 DataTable Component

Verify `frontend/components/shared/DataTable.tsx` has:

- Sortable columns
- Pagination support
- Row selection
- Glassmorphism styling

Verification:

Check	File	Expected
GlassPanel exists	`components/shared/GlassPanel.tsx`	Component exported
StatusBadge exists	`components/shared/StatusBadge.tsx`	Component exported
MetricCard exists	`components/war-room/MetricCard.tsx`	Component exported

Check	File	Expected
DataTable exists	`components/shared/DataTable.tsx`	Component exported
CSS variables used	All components	`var(--*)` patterns

TASK 7 — Environment Configuration Verification 🟢

Time Est: 10 min | **Priority:** 🟢 Medium

Verify `.env` file has all required variables.

7.1 Required Environment Variables

Variable	Purpose	Source
`DB_PASSWORD`	PostgreSQL password	`.env`
`JWT_SECRET`	JWT signing key	`.env`
`REDIS_URL`	Redis connection	`.env`
`DATABASE_URL`	PostgreSQL connection	`.env`
`NEXT_PUBLIC_API_URL`	Backend API URL	`.env`
`NEXT_PUBLIC_WS_URL`	WebSocket URL	`.env`

7.2 Verification Script

```
cd backend && python -c "
from app.config import settings
print('[OK] DB_HOST:', settings.DATABASE_URL.split('@')[1].split('/')[0] if '@' in
settings.DATABASE_URL else 'N/A')
print('[OK] REDIS_URL:', settings.REDIS_URL)
print('[OK] JWT_SECRET:', '***' if settings.JWT_SECRET else 'NOT SET')
"
```

Verification:

Check	Command	Expected
`.env` exists	`ls -la .env`	File present
DB password set	Python check	Not empty
JWT secret set	Python check	`***`
Redis URL set	Python check	`redis://...`

TASK 8 — Full Stack Health Check ⚫

Time Est: 10 min | **Priority:** 🔴 Critical

Comprehensive health check of all services.

8.1 Service Status Matrix

Service	Port	Health Check	Expected
PostgreSQL	5432	`pg_isready`	accepting connections
Redis	6379	`redis-cli ping`	PONG
Backend	8000	`/api/v1/system/health`	`{"status": "healthy"}`
Frontend	3000	Browser	Page loads

8.2 Integration Test

```
# 1. Start all services
docker-compose -f docker-compose.dev.yml up -d postgres redis
cd backend && uvicorn app.main:app --reload --port 8000 &
cd frontend && npm run dev &

# 2. Wait for startup
sleep 5

# 3. Test health endpoint
curl -s http://localhost:8000/api/v1/system/health | python -m json.tool

# 4. Test frontend
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000
```

Expected Output:

```
{
  "status": "healthy",
  "version": "0.1.0",
  "services": {
    "database": {"status": "connected"},
    "redis": {"status": "connected", "latency_ms": 0.xxx}
  }
}
```

```
200
```

Verification:

Check	Command	Expected
All containers up	`docker-compose ps`	2 containers
Backend healthy	`curl /health`	`{"status": "healthy"}`
Frontend accessible	`curl localhost:3000`	HTTP 200
Database connected	Health check	`"database": {"status": "connected"}`
Redis connected	Health check	`"redis": {"status": "connected"}`

Files Created/Modified Today

```
threatmatrix-ai/
├── docs/
│   └── worklog/
│       └── DAY_05_MAR01.md              🔨 This file
├── backend/
│   └── (verification only — no new files)
├── frontend/
│   └── components/
│       └── shared/
│           └── (verify existing components)
└── .env                                 🔨 Verify configuration
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Command	Expected Result
1	Docker running	`docker-compose ps`	2 containers Up
2	PostgreSQL ready	`pg_isready`	accepting connections
3	Redis responding	`redis-cli ping`	PONG
4	Database connected	Python test	`[OK] PostgreSQL version:`
5	Tables exist	Python test	`[OK] Tables found: 10`
6	Migrations applied	`alembic current`	`20260226_000000 (head)`
7	Redis get/set	Python test	`[OK] Redis get/set: hello`
8	Redis pub/sub	Python test	`[OK] Redis pub/sub: subscribed`
9	Backend starts	`uvicorn`	No errors
10	Health endpoint	`curl /health`	`{"status": "healthy"}`
11	OpenAPI docs	Browser `/docs`	Swagger UI loads

#	Verification	Command	Expected Result
12	Frontend starts	`npm run dev`	No errors
13	API connection	Browser console	`[OK] Backend connected`
14	Design components	File check	All 4 components exist
15	Env variables	Python check	All required vars set

Scope Adherence Verification

Requirement	Source	Verification
Docker Compose	MASTER_DOC_PART2 §3.2	`docker-compose ps` shows services
PostgreSQL 16	MASTER_DOC_PART2 §8.1	`SELECT version()` shows 16.x
Redis 7	MASTER_DOC_PART2 §8.1	`redis-cli info server` shows 7.x
Alembic migrations	MASTER_DOC_PART5 §6.1	`alembic current` shows head
FastAPI auto-docs	MASTER_DOC_PART2 §5.1	`/docs` accessible
Glassmorphism UI	MASTER_DOC_PART3 §1.4	GlassPanel has blur
Design system CSS	MASTER_DOC_PART3 §1.2	CSS variables present
Health check	MASTER_DOC_PART2 §5.1	`/system/health` returns status

Blockers

Blocker	Severity	Mitigation	Status
Next.js 16 build error	Medium	Use `npm run dev` for development	Known bug
No LLM API keys yet	Low	Mock responses for now	Expected
No threat intel keys yet	Low	Mock data for now	Expected

Tomorrow's Preview (Day 6)

- Flow service implementation (CRUD endpoints)
- Alert service implementation (lifecycle management)
- WebSocket server setup
- Frontend hooks: useFlows, useAlerts
- War Room components: ThreatMap, LiveAlertFeed

Reference Documents

Document	Section	Purpose

Document	Section	Purpose
MASTER_DOC_PART2	§3.2	Docker Compose service definitions
MASTER_DOC_PART2	§4.2	Database schema (10 tables)
MASTER_DOC_PART2	§5.1	API endpoints (40+ total)
MASTER_DOC_PART2	§6.1	Redis pub/sub architecture
MASTER_DOC_PART3	§1.4	Component patterns (GlassPanel)
MASTER_DOC_PART5	§6.1	Development environment setup
MASTER_DOC_PART5	§6.3	Docker Compose configuration

Task workflow for Day 5 — ThreatMatrix AI Sprint 1